ЛАБОРАТОРНА РОБОТА № 1
СПАДКУВАННЯ І ПОЛІМОРФІЗМ У C#

Мета роботи: набуття вмінь і навичок роботи з конструювання ієрархії класів та використання віртуальних функцій.

Призначення: застосування динамічного поліморфізму, розуміння відмінності між раннім і пізнім зв'язуванням.

Основні теоретичні відомості

Спадкування

Спадкування (inheritance) є одним із принципів ООП. Базовий клас задає загальні ознаки і загальну поведінку для класів-спадкоємців.

Загальні властивості і методи унаслідуються від базового класу. До них додаються і визначаються **нові** властивості і методи.

Таким чином, перш за все спадкування реалізує механізми розширення базового класу.

Нехай у нас є наступний клас Person, який описує окрему людину:

```
class Person
{
    private string _name;

    public string Name
    {
        get { return _name; }
        set { _name = value; }
    }
    public void Display()
    {
        Console.WriteLine(Name);
    }
}
```

Але раптом нам знадобився клас, що описує співробітника підприємства – клас Employee. Оскільки цей клас буде реалізовувати той же функціонал, що і клас Person, тому що співробітник – це також і людина, то було б раціонально зробити клас Employee похідним (або спадкоємцем, або підкласом) від класу

Person, який, в свою чергу, називається базовим класом або батьком (або суперкласом):

```
class Employee : Person
{

}
```

Після двокрапки ми вказуємо базовий клас для даного класу. Для класу Employee базовим є Person, і тому клас Employee успадковує всі ті ж властивості, методи, поля, які є в класі Person. Інша справа, що скористатися в похідному класі можна не всім спадком. Об'єкт-представник похідного класу **не** може отримати доступ до **private** членів базового класу. Використовуємо ще один модифікатор доступу – **protected**. Цей модифікатор забезпечує відкритий доступ до членів базового класу, але тільки для похідного класу.

Віртуальні методи і властивості

При наслідуванні нерідко виникає необхідність змінити в класі-спадкоємці функціонал методу, який був успадкований від базового класу. В цьому випадку клас-спадкоємець може перевизначати методи і властивості базового класу.

Ті методи і властивості, які ми хочемо зробити доступними для перевизначення, в базовому класі позначається модифікатором **virtual**. Такі методи і властивості називають віртуальними.

А щоб перевизначити метод в класі-спадкоємці, цей метод визначається з модифікатором **override**. Перевизначений метод в класі-спадкоємці повинен мати той же набір параметрів, що і віртуальний метод в базовому класі.

Наприклад, розглянемо наступні класи:

```
class Person
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public Person(string firstName, string lastName)
    {
        FirstName = firstName;
        LastName = lastName;
    }
```

```
        public virtual void Display()
        {
            Console.WriteLine($"{FirstName} {LastName}");
        }
    }

    class Employee : Person
    {
        public string Company { get; set; }
        public Employee(string firstName, string lastName, string
company)
            : base(firstName, lastName)
        {
            Company = company;
        }
    }
```

Тут клас Person представляє людину. Клас Employee успадковується від Person і представляє співробітника підприємства. Цей клас крім успадкованих властивостей має ще одну властивість – Company.

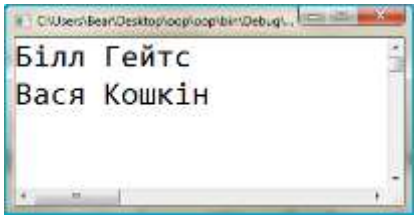
Щоб зробити метод Display доступним для перевизначення, цей метод визначений з модифікатором virtual. Тому ми можемо перевизначити цей метод, але можемо і не перевизначати. Припустимо, нас влаштовує реалізація методу з базового класу. В цьому випадку об'єкти Employee використовуватимуть реалізацію методу Display з класу Person:

```
      static void Main(string[] args)
        {
            Person p1 = new Person("Білл", "Гейтс");
            p1.Display(); // виклик методу Display з класу Person

            Employee p2 = new Employee("Вася", "Кошкін",
"Microsoft");
            p2.Display(); // виклик методу Display з класу Person

            Console.ReadKey();
        }
```

Консольний вивід:

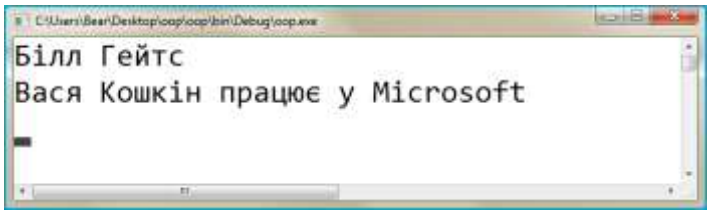


Але також можемо перевизначити віртуальний метод. Для цього в класі-спадкоємці визначається метод з модифікатором **override**, який має таке ж ім'я і набір параметрів:

```
class Employee : Person
{
    public string Company { get; set; }
    public Employee(string firstName, string lastName, string
company)
        : base(firstName, lastName)
    {
        Company = company;
    }

    public override void Display()
    {
        Console.WriteLine
        ($"{FirstName} {LastName} працює у {Company}");
    }
}
```

Для тих же самих об'єктів консольний вивід такий:



Віртуальність успадковується. Віртуальні методи базового класу визначають інтерфейс всієї ієрархії, тобто в будь-якому похідному класі, який не є прямим спадкоємцем від базового класу, можна перевизначити віртуальні методи. Наприклад, ми можемо визначити клас Manager, який буде похідним від Employee, і в ньому також перевизначити віртуальний метод Display класу Employee.

При перевизначенні віртуальних методів слід враховувати ряд обмежень:

Віртуальний і перевизначений методи повинні мати один і той же модифікатор доступу. Тобто, якщо віртуальний метод визначений за допомогою модифікатора public, то і перевизначений метод також повинен мати модифікатор public.

Не можна перевизначити або оголосити віртуальним статичний метод.

Різниця перевизначення і приховування методів

Раніше було розглянуто два способи зміни функціональності методів, успадкованих від базового класу - приховування і перевизначення. У чому різниця між двома цими способами?

Перевизначення

Візьмемо приклад з перевизначенням методів:

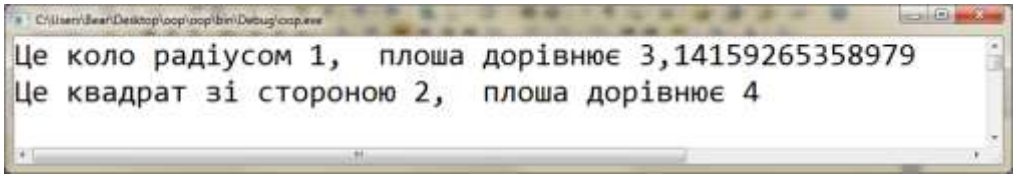
```
class Shape
{
   public double Area { get; set; }
   public virtual void GetInfo()
    {
        Console.WriteLine($"Це просто якась фігура");
    }
}
class Circle : Shape
{
    public double R { get; set; }
    public Circle(double R)
    {
        this.R = R;
        Area = Math.PI * R * R;
    }
    public override void GetInfo()
    {
        Console.WriteLine($"Це коло радіусом {R},  площа
{Area}");
    }
}
class Square : Shape
{
    public double a { get; set; }
    public Square(double a)
    {
        this.a = a;
        Area = a*a;
    }
    public override void GetInfo()
    {
      Console.WriteLine($"Це квадрат зі стороною {a}, площа
{Area}");
    }
}
```

Використовуємо ці класи в програмі в методі Main:

```
class Program
{
    static void Main(string[] args)
    {
        Shape s = new Circle(1);
        s.GetInfo(); // виклик поліморфного методу
        s = new Square(2);
        s.GetInfo(); // виклик поліморфного методу

        Console.ReadKey();
    }
}
```

Результат роботи програми:



Перевизначення методу служить підставою для втілення одного з найефективніших в C# принципів: динамічної диспетчеризації методів, яка представляє собою механізм дозволу виклику під час виконання, а не компіляції.

Значення динамічної диспетчеризації методів полягає в тому, що саме завдяки їй в C # реалізується **динамічний поліморфізм**.

Приховування

Тепер візьмемо ті ж класи, але замість перевизначення використовуємо приховування:

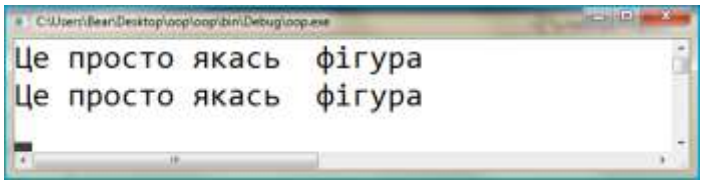
```
class Shape
{
   public double Area { get; set; }
   public virtual void GetInfo()
    {
        Console.WriteLine($"Це просто якась  фігура");
    }
}
class Circle : Shape
{
    public double R { get; set; }
    public Circle(double R)
    {
```

```
            this.R = R;
            Area = Math.PI * R * R;
        }
        public new void GetInfo()
        {
            Console.WriteLine($"Це коло радіусом {R},  площа
{Area}");
        }
    }
    class Square : Shape
    {
        public double a { get; set; }
        public Square(double a)
        {
            this.a = a;
            Area = a*a;
        }
        public new void GetInfo()
        {
         Console.WriteLine($"Це квадрат зі стороною {a},  площа
{Area}");
        }
    }
```

І подивимося, що буде в наступному випадку:

```
        static void Main(string[] args)
        {
            Shape s = new Circle(1);
            s.GetInfo(); // метод вже не поліморфний!
            s = new Square(2);
            s.GetInfo(); // метод вже не поліморфний!
            Console.ReadKey();
        }
```

Метод Main точно такий самий, як і в попередньому прикладі, але результат зовсім інший:



Змінна s представляє тип Shape, але зберігає посилання на об'єкт Circle і Square. Однак при виклику методу GetInfo буде виконуватися та версія методу, яка визначена саме в класі Shape, а не в класах Circle і Square. Чому? Класи Circle і Square ніяк не переважають метод GetInfo, успадкований від базового

класу, а фактично визначає новий метод. Тому при виклику s.GetInfo(); викликається метод GetInfo з класу Shape.

Що дає перевизначення методів

Завдяки перевизначенню методів в C# підтримується **динамічний поліморфізм.**

В об'єктно-орієнтованому програмуванні поліморфізм грає дуже важливу роль, тому що він дозволяє визначити в загальному класі методи, які стають загальними для всіх похідних від нього класів, а в похідних класах – визначити конкретну реалізацію деяких або ж усіх цих методів.

Перевизначення методів – це ще один спосіб втілити в C# головний принцип поліморфізму: **один інтерфейс – багато методів.**

Завдання до лабораторної роботи

Описати базовий клас (можливо, абстрактний), у якому за допомогою віртуальних або абстрактних методів та властивостей задається інтерфейс для похідних класів.

В усіх класах потрібно перевизначити метод Equals, щоби порівнювались значення, а не посилання. Визначити в класах усі можливі конструктори.

Метод Main повинен утримувати масив або список з елементів базового класу, заповнений посиланнями на похідні класи, та демонструвати використання усіх розроблених елементів класів відповідно варіанту.

Показати на прикладі одного з методів, присутніх у кожному класі, використання поліморфізму.

Обов’язково продемонструвати різницю між раннім та пізнім зв’язуванням

Варіанти завдань

1. Трикутник, геометрічна фігура, трапеція, рівносторонній трикутник. Поліморфний метод повертає значення площі відповідної фігури.

2. Куля, усічений прямий конус, геометрічне тіло, конус. Поліморфний метод повертає значення об’єму відповідного геометрічного тіла. (Формули для обчислення знайти самостійно)

3. Куля, геометрічне тіло, паралелепіпед, тетраедр. Поліморфний метод повертає значення площі поверхні тіла.

4. Кінь, слон, тура, ферзь, шахова фігура. Поліморфний метод приймає два параметра: перший-позиція фігури на шахівниці (наприклад, g2), а другий параметр – поле, куди треба піти (наприклад, h3), і виводить повідомлення – чи можливий такий хід, чи ні;

5. Кінь, слон, тура, ферзь, шахова фігура. Поліморфний метод приймає два параметра: перший – поле, де стоїть король противника (наприклад, g2), а другий параметр – позиція фігури на шахівниці (наприклад, h3), і виводить повідомлення – чи оголошений королю шах, чи ні;

6. Організація, страхова компанія, будівна компанія, метробуд. Поліморфний метод для страхової компанії виводить прізвище страхового агента, кількість полісів і загальну суму, для будівної компанії - прізвище прораба, загальну смету (суму в грн) і метраж (загальну площу кв.м), для метробуд – назва маршруту, кількість пасажирів, відстань в км.;

7. Ціле беззнакове число, двійкове число, десяткове число, шістнадцяткове число. Поліморфний метод виводить на екран число відповідно як двійкове, десяткове або шістнадцяткове.

8. Базовий клас – трикутник (поля – довжини двох сторін і кут між ними), похідні – рівносторонній, рівнобедрений, прямокутний. Поліморфний метод виводить на екран значення площі та периметра.

9. Двигун, вічний двигун (perpetuum mobile), двигун внутрішнього згоряння, електродвигун. Поліморфний метод для двигуна внутрішнього згоряння виводить робочий об'єм двигуна (л), потужність (л.с) і витрата бензину на 100 км., для електродвигуна - потужність (квт), напруга (в) і

кількість фаз, ну а для perpetuum mobile - придумайте самі, щоб показати, що такий не існує.

10. Опір, активний опір, ємнісний опір, індуктивний опір. Поліморфна функція повертає значення опіру для відповідного типу і виводить повідомлення про тип опіру. Ланцюг змінного струму включає в себе певну кількість активних, ємнісних і індуктивних елементів.

активний опір $R = \frac{\rho l}{S}$; де р - питомий опір провідника, l - довжина провідника, S - площа перерізу провідника;

ємнісний опір $X_c = \frac{1}{\omega C}$; де ω -циклічна частота змінного струму (рад/с), C – ємність;

індуктивний опір $X_L = \omega L$, де L - індуктивність.

11. Базовий клас – Тіло, похідні – Куля, Паралелепіпед, Тетраедр. Обов’язкові методи – обчислення площі поверхні та об’єму. Архімед кинув у повну до країв ванну золоту кулю, тетраєдр і паралелепіпед. Який об’єм води витече?

Контрольні питання

1. Який модифікатор доступу дозволяє тільки членам класів-спадкоємців звертатися до поля базового класу?
2. На чому базується динамічний поліморфізм?
3. Скільки може бути базових класів у похідного класа C#?
4. Скільки об’єктів може бути створено від абстрактного класу?
5. Чи успадковується конструктор?
6. Чи може бути віртуальний конструктор?
7. У чому різниця між раннім та пізнім зв’язуванням?